# Python for Artificial Intelligence

## Object-Oriented Programming I: Classes & Objects

*Week 2, Day 3*

# Today's Agenda

- Hour 1: Classes, objects, attributes, and the __init__ constructor
- Hour 2: Methods, self, instance attributes vs. class attributes
- Hour 3: Practice modeling real-world entities as classes
- By the end: you can design your own data type that bundles data and behavior together

# Hour 1

Classes, Objects & __init__

EXAMPLE

# The Procedural Way: A Bank Account Program

*Two customers, each balance tracked as its own separate variable.*

```
ali_balance = 1000
sara_balance = 500

def deposit(balance, amount):
    return balance + amount

def withdraw(balance, amount):
    if amount > balance:
        print("Insufficient funds")
        return balance
    return balance - amount

ali_balance = deposit(ali_balance, 200)
sara_balance = withdraw(sara_balance, 100)

print(ali_balance)   # 1200
print(sara_balance)  # 400
```

CONCEPT

# What's Wrong With This?

- Every customer needs a brand new variable: ali_balance, sara_balance, zain_balance... this does not scale to thousands of customers
- Nothing links a balance to its owner's name, account number, or account type - that information lives somewhere else entirely
- It's easy to update the wrong variable by mistake: sara_balance = withdraw(ali_balance, 100) would silently mix up Sara's and Ali's money
- Every function call needs the right balance passed in and the result reassigned every time; one missed reassignment and the change is silently lost
- The data (balances) and the behavior (deposit, withdraw) are never bundled together - they just happen to be used near each other

EXAMPLE

# The Class Fix: Bundling Balance and Behavior Together

*Don't worry about the exact syntax yet, we'll break it down piece by piece right after this. For now, just see what changed.*

```
class BankAccount:
    def __init__(self, owner, balance):
        self.owner = owner
        self.balance = balance

    def deposit(self, amount):
        self.balance = self.balance + amount
    def withdraw(self, amount):
        if amount > self.balance:
            print("Insufficient funds")
        else:
            self.balance = self.balance - amount
ali = BankAccount("Ali", 1000)
sara = BankAccount("Sara", 500)
ali.deposit(200)
sara.withdraw(100)
print(ali.balance, sara.balance)  # 1200 400
```

CONCEPT

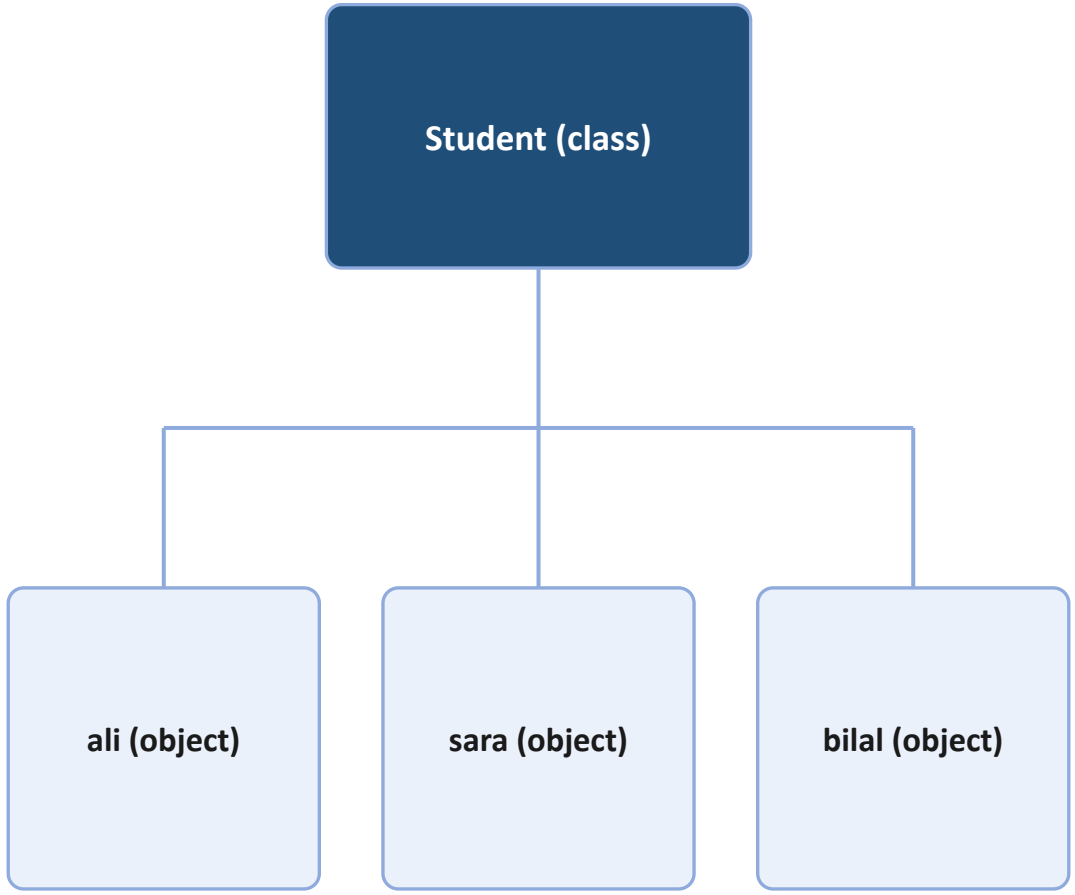
# Why Do We Need Classes?

- The BankAccount version fixed three things at once: one object per customer, balance and actions bundled together, and no risk of mixing up whose balance you're changing
- A class is a blueprint for creating your own data type, one that bundles data (attributes) with behavior (methods)
- An object (or instance) is one specific thing built from that blueprint
- Example: 'BankAccount' is the blueprint (class); 'ali, balance 1000' is one actual account (object)

CONCEPT

# Class vs. Object

- The class defines what every object of that type will have and can do
- Every object created from the same class has the same attributes, but each has its own values
- You can create as many objects from one class as you need
- This mirrors real life: 'Car' is a category; your specific car is one instance of that category



# Class Definition & __init__ Syntax

**SYNTAX**

```
class ClassName:
    def __init__(self, param1, param2):
        self.attribute1 = param1
        self.attribute2 = param2
```

**PARAMETERS**

**`self`** refers to the specific object being created or used; always the first parameter

**`__init__`** a special method that runs automatically when a new object is created

**`self.attribute1`** stores a value on this specific object, so it can be read back later

**YOU CAN**

✓ Create an object with ClassName(value1, value2)

✓ Give each object different attribute values

✓ Access an attribute later with object_name.attribute1

*Returns: nothing; __init__ sets up the object but does not return a value*

**YOU CANNOT**

✗ Call __init__() directly yourself; Python calls it automatically

✗ Forget self as the first parameter in __init__

EXAMPLE

# Defining a Simple Student Class

*__init__ runs automatically and sets up name and age on each new object.*

```
class Student:
    def __init__(self, name, age):
        self.name = name
        self.age = age

ali = Student("Ali", 20)
sara = Student("Sara", 22)

print(ali.name, ali.age)    # Ali 20
print(sara.name, sara.age)  # Sara 22
```

THINK & PRACTICE

# Quick Check

Define a class Book with __init__ taking title and author. Create two Book objects with different titles and authors, then print both titles.

ANSWER

# Quick Check: Answer

```
class Book:
    def __init__(self, title, author):
        self.title = title
        self.author = author

book1 = Book("Dune", "Frank Herbert")
book2 = Book("1984", "George Orwell")

print(book1.title)  # Dune
print(book2.title)  # 1984
```

CONCEPT

# Types of Constructors

- __init__ is always called a 'constructor' - but it can be written in a few different styles
- Default constructor: takes no parameters beyond self; every object gets the same hardcoded starting values
- Parameterized constructor: takes parameters and uses them to set each object's attributes (what you just learned)
- Constructor with default parameters: parameters have default values, so you can create an object with or without supplying them
- These are not different features, just different ways of writing the same __init__ method

EXAMPLE

# Default Constructor

*No parameters beyond self; every object starts identical.*

```
class Student:
    def __init__(self):
        self.name = "Unknown"
        self.age = 0

s = Student()
print(s.name, s.age)  # Unknown 0
```

EXAMPLE

# Constructor with Default Parameters

*Combines the last two Days' worth of ideas: default arguments (Day 6) applied to __init__.*

```
class Student:
    def __init__(self, name="Unknown", age=0):
        self.name = name
        self.age = age

s1 = Student()                # uses both defaults
s2 = Student("Ali", 20)      # overrides both
s3 = Student(name="Sara")    # overrides just one

print(s1.name, s1.age)  # Unknown 0
print(s2.name, s2.age)  # Ali 20
print(s3.name, s3.age)  # Sara 0
```

THINK & PRACTICE

# Quick Check

Rewrite the Book class's __init__ so title defaults to "Untitled" and author defaults to "Unknown". Then create a Book with no arguments, and print its title and author.

ANSWER

# Quick Check: Answer

```
class Book:
    def __init__(self, title="Untitled", author="Unknown"):
        self.title = title
        self.author = author

b = Book()
print(b.title, b.author)  # Untitled Unknown
```

CONCEPT

# A Quick Note: Destructors

- If __init__ runs when an object is created, __del__ runs when an object is destroyed - it's called a destructor
- Used for cleanup: closing a file, releasing a connection, logging that something ended
- Python cleans up memory automatically (garbage collection), so __del__ is far less commonly needed than in languages like C++
- Still worth knowing the vocabulary: constructor creates, destructor cleans up

EXAMPLE

# __init__ and __del__ Together

```
class Student:
    def __init__(self, name):
        self.name = name
        print(self.name, "enrolled")

    def __del__(self):
        print(self.name, "record removed")

s = Student("Ali")    # Ali enrolled
del s                  # Ali record removed
```

CONCEPT

# Attributes Can Be Changed After Creation

- Once an object exists, you can read its attributes with object.attribute
- You can also update an attribute's value with object.attribute = new_value
- Each object's attributes are independent; changing sara's age does not affect ali's age
- This independence is the whole point: one blueprint, many separately-tracked objects

EXAMPLE

# Reading and Updating Attributes

```
ali = Student("Ali", 20)

print(ali.age)  # 20
ali.age = 21
print(ali.age)  # 21

sara = Student("Sara", 22)
print(sara.age)  # 22  (unaffected by ali's change)
```

THINK & PRACTICE

# Think & Practice

Using the Book class from before, create a book object, then update its author attribute to a new value and print the updated author.

ANSWER

# Think & Practice: Answer

```
book1 = Book("Dune", "Frank Herbert")
print(book1.author)  # Frank Herbert

book1.author = "F. Herbert"
print(book1.author)  # F. Herbert
```

# Hour 2

Methods, self & Class Attributes

CONCEPT

# Methods: Functions That Belong to a Class

- A method is a function defined inside a class; it describes something objects of that class can do
- Every method's first parameter is self, which refers to the specific object calling it
- Inside a method, self.attribute reads or changes that object's own data
- You call a method with object_name.method_name(), Python passes self automatically

# Defining and Calling a Method

**SYNTAX**

```
class ClassName:
    def method_name(self, param):
        # use self.attribute here
        return something

object_name.method_name(value)
```

**PARAMETERS**

**`self`** the specific object the method was called on; Python supplies this automatically

**`param`** any additional value the method needs, just like a regular function parameter

**YOU CAN**

✓ Read or update self.attribute inside the method

✓ Call other methods on the same object using self.other_method()

✓ Return a value from a method, just like a normal function

**YOU CANNOT**

✗ Call the method without an object (Student.study() has no object to work with)

✗ Forget self as the first parameter when defining the method

*Returns: whatever the method's return statement specifies, or nothing if there isn't one*

EXAMPLE

# Adding a Method to Student

*study() uses self.name to build a personalized message.*

```
class Student:
    def __init__(self, name, age):
        self.name = name
        self.age = age

    def introduce(self):
        return self.name + " is " + str(self.age) + " years old."

ali = Student("Ali", 20)
print(ali.introduce())  # Ali is 20 years old.
```

CONCEPT

# Instance Attributes vs. Class Attributes

- An instance attribute belongs to one specific object; it is set with self.attribute inside __init__
- A class attribute belongs to the class itself, shared by every object made from it
- Class attributes are written directly inside the class body, not inside a method
- Good use case: a school_name that is the same for every Student object

**Instance attribute**

```
class Student:
    def
__init__(self,
name):
        self.name =
name

# each object has
# its own name
```



**Class attribute**

```
class Student:
    school = "City
High"

# every object
shares
# the same school
value
```

EXAMPLE

# Using a Class Attribute

*school is defined once on the class and shared by every Student object.*

```
class Student:
    school = "City High"

    def __init__(self, name):
        self.name = name

    def introduce(self):
        return self.name + "studies at" + self.school

ali = Student("Ali")
sara = Student("Sara")

print(ali.introduce())   # City High
print(sara.introduce())  # City High
```

THINK & PRACTICE

# Quick Check

Add a method total_price(self) to a Product class that returns self.price * self.quantity. Assume __init__ already sets self.price and self.quantity.

ANSWER

# Quick Check: Answer

```
class Product:
    def __init__(self, price, quantity):
        self.price = price
        self.quantity = quantity

    def total_price(self):
        return self.price * self.quantity

item = Product(50, 3)
print(item.total_price())  # 150
```

THINK & PRACTICE

# Think & Practice (Harder)

Create a class BankAccount with __init__ setting self.balance = starting_balance. Add a method deposit(self, amount) that adds amount to self.balance, and a method withdraw(self, amount) that subtracts amount only if there is enough balance, otherwise it should return the string "Insufficient funds".

ANSWER

# Think & Practice (Harder): Answer

```
class BankAccount:
    def __init__(self, starting_balance):
        self.balance = starting_balance

    def deposit(self, amount):
        self.balance = self.balance + amount

    def withdraw(self, amount):
        if amount > self.balance:
            return "Insufficient funds"
        self.balance = self.balance - amount

acc = BankAccount(100)
acc.deposit(50)
print(acc.balance)          # 150
print(acc.withdraw(1000))   # Insufficient funds
```

# 15-Minute Break

Stretch, grab a coffee. We'll resume shortly.

# Hour 3

Practice: Modeling Real-World Entities

CONCEPT

# Designing a Class: Ask Two Questions

- 1. What data does one of these things need to remember? These become your __init__ attributes
- 2. What can one of these things do? These become your methods
- Start simple: 2-4 attributes and 1-3 methods is a solid first version of any class
- You can always add more attributes and methods later, once the basics work

WORKED EXAMPLE

# Modeling a BankAccount End to End

*Putting attributes and methods together into one working class.*

```
class BankAccount:
    def __init__(self, owner, starting_balance):
        self.owner = owner
        self.balance = starting_balance

    def deposit(self, amount):
        self.balance += amount

    def withdraw(self, amount):
        if amount > self.balance:
            return "Insufficient funds"
        self.balance -= amount

    def summary(self):
        return self.owner + "'s balance: " + str(self.balance)

acc = BankAccount("Ayesha", 500)
acc.deposit(200)
print(acc.summary())  # Ayesha's balance: 700
```

THINK & PRACTICE

# Practice

Design a class Movie with __init__ setting title, director, and rating. Add a method is_highly_rated(self) that returns True if self.rating is 8 or above, otherwise False. Create two Movie objects and test the method on both.

ANSWER

# Practice: Answer

```
class Movie:
    def __init__(self, title, director, rating):
        self.title = title
        self.director = director
        self.rating = rating

    def is_highly_rated(self):
        return self.rating >= 8

m1 = Movie("Inception", "Christopher Nolan", 8.8)
m2 = Movie("Cats", "Tom Hooper", 2.8)

print(m1.is_highly_rated())  # True
print(m2.is_highly_rated())  # False
```

THINK & PRACTICE

# Practice (Harder)

Design a class Student with __init__ setting name and a list of grades (a list of numbers). Add a method average(self) that returns the average of self.grades, and a method passed(self) that returns True if average() is 40 or above.

ANSWER

# Practice (Harder): Answer

```
class Student:
    def __init__(self, name, grades):
        self.name = name
        self.grades = grades

    def average(self):
        return sum(self.grades) / len(self.grades)

    def passed(self):
        return self.average() >= 40

s = Student("Zara", [55, 60, 30, 70])
print(s.average())  # 53.75
print(s.passed())   # True
```

# Home Practice: Interview-Style Questions

- 1. What is the difference between a class and an object? Give your own example.
- 2. What does self refer to inside a method, and why is it always the first parameter?
- 3. Design a class Rectangle with width and height attributes, and a method area(self).
- 4. What is the difference between an instance attribute and a class attribute?
- 5. Explain what __init__ does and when it runs.
- 6. Name the three constructor styles you learned and write one line of code for each.
- 7. What does __del__ do, and why is it rarely needed in Python compared to C++?

# Key Takeaways & Coming Up

- A class is a blueprint; an object is one specific instance built from that blueprint
- __init__ runs automatically when an object is created; it can be default, parameterized, or use default parameter values
- __del__ (destructor) runs automatically when an object is destroyed, mainly used for cleanup
- Methods are functions inside a class; self always refers to the specific object calling the method
- Instance attributes belong to one object; class attributes are shared by every object of that class
- Coming up in Day 9: Inheritance and polymorphism, building classes on top of other classes